PQC DAILY STUDY SERIES

PQC Standards Showdown: ML-KEM vs. ML-DSA vs. SLH-DSA

Day 42 | Category: PQC Foundational Concepts Date: 2026-10-27 | DAILY MONITORING

PQC Standards Showdown: ML-KEM vs. ML-DSA vs. SLH-DSA

After deep dives into the individual NIST PQC standards, it's time for a comparative analysis. Understanding the trade-offs between ML-KEM (CRYSTALS-Kyber), ML-DSA (CRYSTALS-Dilithium), and SLH-DSA (SPHINCS+) is crucial for engineers and security architects who will implement them. These algorithms serve different cryptographic purposes and have vastly different performance characteristics.

Core Cryptographic Function

> The most fundamental difference lies in their cryptographic function. One is for establishing secrets, while the other two are for verifying authenticity.

- **ML-KEM (Kyber):** A **Key Encapsulation Mechanism (KEM)**. Its purpose is to securely establish a shared secret between two parties over an insecure channel. It is not used for signing. In internet protocols, it replaces elliptic-curve Diffie-Hellman (ECDH).
- **ML-DSA (Dilithium):** A **Digital Signature Algorithm (DSA)**. Its purpose is to create a digital signature to verify the authenticity and integrity of a message or data. It replaces algorithms like RSA, ECDSA, and EdDSA for signing certificates and messages.
- **SLH-DSA (SPHINCS+):** Also a **Digital Signature Algorithm (DSA)**. Like ML-DSA, it is used for digital signatures. However, its security is based on different mathematical principles (hash functions vs. structured lattices), leading to a different performance profile.

Key Metrics and Performance Comparison

Performance is a critical factor for real-world applicability, especially in latency-sensitive internet protocols. The table below summarizes key metrics for the recommended security level (NIST Level 1/2).

Metric	ML-KEM (Kyber-768)	ML-DSA (Dilithium-2)	SLH-DSA (SPHINCS+-128s)
Primary Use Case	Key Exchange (e.g., TLS, VPNs)	General-Purpose Signatures (e.g., X.509, JWTs)	High-Assurance Signatures (e.g., Firmware, CAs)
Underlying Problem	Module Learning with Errors (MLWE)	Module Learning with Errors (MLWE)	Security of Hash Functions
Public Key Size	~1,184 bytes	~1,312 bytes	~32 bytes (very small!)
Private Key Size	~2,400 bytes	~2,528 bytes	~64 bytes (very small!)
Signature/Ciphertext Size	~1,088 bytes (Ciphertext)	~2,420 bytes (Signature)	~7,856 bytes (very large!)
Signing Speed	N/A (Encapsulation is fast)	Very Fast	Extremely Slow
Verification Speed	N/A (Decapsulation is fast)	Fast	Fast

Key Takeaways from the Comparison:

- **Lattice-based Dominance (ML-KEM/ML-DSA):** For most common use cases like TLS handshakes and general code signing, the CRYSTALS family offers a balanced and highly efficient profile with small keys and fast operations.
- **SPHINCS+ for Conservative Trust:** SLH-DSA's strength is its minimal trust assumption—it only relies on the security of its underlying hash function. This makes it an excellent choice for offline, high-assurance scenarios like root CA signing or firmware signing, where its slow signing speed and large signature size are acceptable trade-offs for its conservative security posture.
- **Size Matters:** The most significant practical difference is the signature size. An ML-DSA signature is ~2.4 KB, while an SLH-DSA signature is nearly 8 KB. For protocols that transmit many signatures, this has a direct impact on bandwidth.

Applicability to Internet Protocols

- **TLS 1.3:** ML-KEM is the primary choice for the post-quantum key exchange mechanism, often in a hybrid mode with X25519 initially. ML-DSA will be used to sign the certificates presented by servers.
- **SSH & VPNs (WireGuard, IKEv2):** Similar to TLS, ML-KEM is used for establishing the secure channel, while ML-DSA is used to authenticate the server/peers.

  - **Code Signing & Software Updates:** ML-DSA is the front-runner for signing software and firmware due to its speed. SLH-DSA may be used for the root of trust or critical firmware where verification speed is more important than signing speed.

Daily Quiz

1. You are designing a secure chat application that needs to establish encrypted sessions. Which PQC algorithm is the appropriate choice for this task?
2. A Certificate Authority (CA) needs to sign a root certificate that will be valid for decades. They are most concerned about the long-term security and. conservative nature of the algorithm, and signing speed is not a concern. Which algorithm is the most suitable?
3. Why is ML-KEM not used for signing digital documents?
